**Subject-**MlOps, Assignmnet-2**, Roll No-**G25AIT2131**, Name-**Mahadev Vishwakarma

---

**Final Report - Goodreads Genre Classification using DistilBERT**

---

## 1. Introduction

This assignment focused on building a Natural Language Processing (NLP) model for Goodreads book genre classification using transformer-based deep learning techniques. The implementation used the Hugging Face Transformers library along with PyTorch inside a Kaggle Notebook environment. The workflow included data preprocessing, tokenization, model training, evaluation, experiment tracking, and model deployment.

The selected model was DistilBERT, a lightweight transformer architecture optimized for faster training while maintaining strong performance on NLP tasks. The assignment also incorporated modern MLOps practices such as secure secret management, experiment logging, and cloud-based GPU training

## 2. Model Selection Rationale (why did I choose this model?)

The model selected for this assignment was **DistilBERT (distilbert-base-cased)**. DistilBERT is a compressed and optimized version of the original BERT model developed by Google. It retains most of BERT's language understanding capabilities while being smaller, faster, and more computationally efficient.

The Goodreads dataset contains book reviews with rich contextual and semantic information. Traditional machine learning techniques such as TF-IDF with Logistic Regression or Naive Bayes generally fail to capture deeper relationships between words and sentence structures. Transformer models overcome this limitation using self-attention mechanisms that analyze contextual dependencies between words across the entire sentence.

DistilBERT was selected for several important reasons mentioned below.

### a. Computational Efficiency

Compared to the full BERT architecture, DistilBERT is approximately 40% smaller and significantly faster(60% ). Since Kaggle notebooks provide limited GPU resources, a lightweight transformer was more practical for training within memory and runtime constraints.

### b. Strong NLP Performance

DistilBERT performs exceptionally well on text classification tasks. It preserves around 95–97% of BERT's language understanding capability while reducing computational overhead. This made it suitable for Goodreads review classification.

### c. Transfer Learning Benefits

The model was pre-trained on large-scale text corpora before fine-tuning on the Goodreads dataset. Transfer learning allowed the model to leverage existing language knowledge and achieve good results even with limited training data.

### d. Easy Integration with Hugging Face

The Hugging Face ecosystem simplified implementation through:

- Pre-trained tokenizers
- Trainer APIs
- Dataset utilities
- Built-in evaluation support
- Easy deployment to Hugging Face Hub

### e. GPU Compatibility

DistilBERT required less VRAM compared to larger transformer architectures such as RoBERTa or DeBERTa, making it ideal for Kaggle's free GPU environment.
The final training pipeline included

- DistilBertTokenizerFast
- Hugging Face Trainer
- PyTorch backend
- Accuracy and weighted F1-score evaluation metrics
- WandB experiment logging
- Hugging Face model deployment

## 3. Kaggle Training Setup

The assignment was implemented and trained using a **Kaggle Notebook** environment. Kaggle provides free cloud-based GPUs that are highly useful for machine learning experimentation and deep learning workflows.

### a. Notebook Configuration

The Kaggle notebook was configured with below

- Python environment
- GPU acceleration enabled (Notebook Settings → Accelerator → GPU T4 X 2)
- Internet access enabled (for Hugging Face model downloads)
- PyTorch and Transformers libraries installed

### b. Secure Secret Management

Kaggle Secrets were used to securely store API credentials such as

- Hugging Face access token
- WandB API key

Secrets were accessed using below code

```
from kaggle_secrets import UserSecretsClient
from huggingface_hub import login
import os

secrets = UserSecretsClient()
os.environ["WANDB_API_KEY"] = secrets.get_secret("WANDB_KEY")
huggingface_token = secrets.get_secret("HUGGINGFACE_TOKEN")
login(token=huggingface_token)
```

**c. Training Configuration**

➢ The training configuration included

- Epochs: 3
- Learning rate: 5e-5
- Training batch size: 10
- Evaluation batch size: 16
- Weight decay for regularization
- Warmup steps for optimization stability

➢ The Hugging Face Trainer API automated

- Training loops
- Evaluation
- Checkpointing
- GPU utilization
- Metric logging

➢ The dataset pipeline involved

- Loading Goodreads review datasets
- Text preprocessing
- Tokenization into subword tokens
- Conversion into PyTorch datasets
- Fine-tuning DistilBERT on labeled genres

## 4. Challenges and Learnings (what was hard? What would you do differently?)

Several technical and practical challenges were encountered during this assignment.

**a. Challenges Faced**

➢ **GPU Memory Limitations**

Transformer models are computationally expensive. Long review texts and larger batch sizes caused GPU memory constraints. Batch sizes had to be carefully adjusted to avoid CUDA out-of-memory errors.

➢ **Long Training Time**

Fine-tuning transformer architectures requires substantially more training time than traditional machine learning approaches.

- **Hyperparameter Tuning**

  Selecting suitable learning rates, warmup steps, and batch sizes required experimentation and repeated training runs.

- **Dataset Processing**

  Handling Goodreads review data involved
  - Cleaning noisy text
  - Managing class balance
  - Sampling reviews efficiently
  - Maintaining consistent token lengths

- **MLOps Integration**

Integrating WandB logging and Hugging Face Hub deployment introduced additional configuration complexity, especially regarding authentication and secret management.

### b. Key Learnings

This assignment provided valuable practical experience in
- Transformer-based NLP
- Fine-tuning pre-trained language models
- GPU training workflows
- Experiment tracking using WandB
- Secure secret management in Kaggle
- Hugging Face model deployment
- Cloud-based machine learning workflows

The assignment also improved understanding of transfer learning and modern NLP pipelines.

### c. What would you do differently?

several improvements could be explored with this assignment
- Testing larger transformer models such as RoBERTa or DeBERTa
- More extensive hyperparameter tuning
- Cross-validation for stronger evaluation
- Advanced preprocessing techniques
- Data augmentation methods
- Ensemble-based approaches for improved performance

## 5. Result

| SN | Metric | Score |
|---|---|---|
| 1 | Accuracy | 0.61875 |
| 2 | F1 Score | 0.61643 |
| 3 | Eval Loss | 2.29844 |

## 6. Conclusion

This assignment demonstrated a complete end-to-end NLP workflow using Kaggle infrastructure and Hugging Face tools. DistilBERT provided an effective balance between efficiency and classification performance while remaining suitable for limited GPU environments.

The assignment also strengthened practical MLOps skills including cloud-based training, experiment tracking, secret management, and model deployment. Overall, the assignment provided valuable hands-on experience with modern deep learning and NLP engineering workflows.

## 7. Links

GitHub Repository Link
https://github.com/g25ait2131/MLOps-Assignment2

Kaggle Notebook Link
https://www.kaggle.com/code/mahadevvishwakarma/a2-mlops-2

Hugging Face Model Link
https://huggingface.co/g25ait2131/mlops-assign2

W&B Project Dashboard Link
https://wandb.ai/g25ait2131-iit/MLOps-Assignment2